ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА
Факультет електроніки та комп'ютерних технологій

ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №4

«Професійне ООП та система типів Python»

з курсу «Професійний Python»
Варіант №13 — «Система обліку комп'ютерної техніки»

Виконав: Капарник Дмитро Андрійович

Група: ФЕП-32

Перевірив: доцент Куньо Іван Михайлович

Дата: 18.10.2026

Львів — 2026

1. Мета роботи

Побудувати типізовану об’єктну модель із композицією, успадкуванням, абстракціями та перевіркою типів.

2. Індивідуальний варіант і постановка задачі

Варіант №13 — система обліку комп’ютерної техніки. Реалізувати Device, Computer, Printer, Manufacturer, InventoryRegistry, repository та maintenance/reporting service; використати Generic, TypeVar, TypedDict, ABC, Protocol і dependency injection.

Наскрізний проєкт продовжує ЛР №1. Базові поля: інвентарний номер, тип, виробник, модель і вартість. Збережено пошук, фільтр виробника, середню/загальну вартість і визначення найдорожчого пристрою.

3. Структура проєкту та створені модулі

```
inventory-service/
  pyproject.toml, README.md, .gitignore
  src/computer_inventory/
    models.py
    domain.py
    services.py
    main.py
  src/inventory_service/  # фінальний пакет запуску
  tests/
  scripts/
  docs/GITHUB.md
```

4. Предметні класи та зв’язки

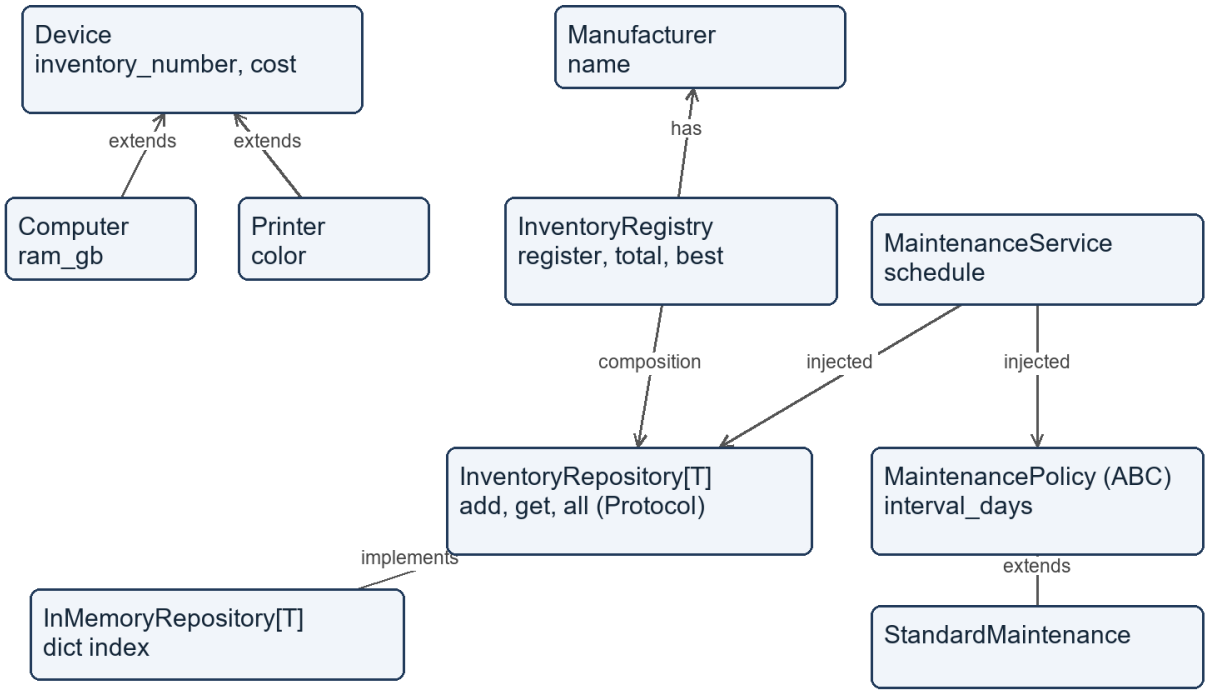


Рисунок 1 — Класи та основні зв’язки предметної моделі

Computer і Printer успадковують Device (is-a). InventoryRegistry містить repository та список Manufacturer (has-a). MaintenanceService отримує репозиторій і політику через конструктор. Money — frozen value object. Інкапсуляція InMemoryRepository приховує словник _items. Властивості registry.total та registry.average повертають обчислені Money.

```
@dataclass(frozen=True)
class Money:
    amount: Decimal
    currency: str = "UAH"

    def __post_init__(self) -> None:
        if not self.amount.is_finite() or self.amount < 0:
            raise ValueError("Invalid money amount")
        if self.currency != "UAH":
            raise ValueError("Only UAH is supported")

    def __add__(self, other: "Money") -> "Money":
        return Money(self.amount + other.amount, self.currency)

    def __str__(self) -> str:
        return f"{self.amount:.2f} {self.currency}"

@dataclass(frozen=True)
class Manufacturer:
    name: str

    def __post_init__(self) -> None:
```

```
        if not self.name.strip():
            raise ValueError("Empty manufacturer")

@dataclass(frozen=True)
class Computer(Device):
    ram_gb: int = 8

    def __post_init__(self) -> None:
        super().__post_init__()
        if self.ram_gb < 1:
            raise ValueError("RAM must be positive")

@dataclass(frozen=True)
class Printer(Device):
    color: bool = False
```

5. Контракти, generic repository та сервіс

```
class InventoryRepository(Protocol[T]):
    def add(self, item: T) -> None: ...
    def get(self, inventory_number: str) -> T | None: ...
    def all(self) -> list[T]: ...

class InMemoryRepository(Generic[T]):
    def __init__(self) -> None:
        self._items: dict[str, T] = {}

    def add(self, item: T) -> None:
        key = item.inventory_number.casefold()
        if key in self._items:
            raise ValueError("Duplicate inventory number")
        self._items[key] = item

    def get(self, inventory_number: str) -> T | None:
        return self._items.get(inventory_number.strip().casefold())

    def all(self) -> list[T]:
        return list(self._items.values())

class MaintenancePolicy(ABC):
    @abstractmethod
    def interval_days(self, device: Device) -> int:
        """Return the recommended maintenance interval."""

class StandardMaintenance(MaintenancePolicy):
    def interval_days(self, device: Device) -> int:
        return 90 if isinstance(device, Printer) else 180

class MaintenanceService:
    def __init__(
        self, repository: InventoryRepository[Device], policy: MaintenancePolicy
    ) -> None:
        self.repository = repository
        self.policy = policy
```

```
    def schedule(self, inventory_number: str) -> int:
        device = self.repository.get(inventory_number)
        if device is None:
            raise LookupError(inventory_number)
        return self.policy.interval_days(device)
```

TypeVar T обмежений Device, тому InMemoryRepository[Device] не приймає Manufacturer. Protocol описує структурну сумісність репозиторію. ABC MaintenancePolicy визначає обов'язковий interval_days. StandardMaintenance задає 90 днів для принтера та 180 для іншого пристрою. Поділ responsibilities реалізує SRP; заміна політики без зміни сервісу — OCP; залежність від контракту й DI — DIP. Принтер і комп'ютер зберігають контракт Device, що підтримує LSP.

6. Колекція, dunder methods і зовнішні дані

```
@dataclass
class InventoryRegistry:
    repository: InventoryRepository[Device]
    manufacturers: list[Manufacturer] = field(default_factory=list)

    def register(self, device: Device) -> None:
        self.repository.add(device)
        manufacturer = Manufacturer(device.manufacturer)
        if manufacturer not in self.manufacturers:
            self.manufacturers.append(manufacturer)

    @property
    def total(self) -> Money:
        return Money(total_cost(self.repository.all()))

    @property
    def average(self) -> Money:
        return Money(average_cost(self.repository.all()))

    def best(self) -> Device | None:
        return most_expensive(self.repository.all())

    def by_manufacturer(self, name: str) -> list[Device]:
        return filter_by_manufacturer(self.repository.all(), name)

    def __len__(self) -> int:
        return len(self.repository.all())

    def __iter__(self) -> Iterator[Device]:
        return iter(self.repository.all())

    def __contains__(self, inventory_number: str) -> bool:
        return self.repository.get(inventory_number) is not None

    def __getitem__(self, inventory_number: str) -> Device:
        device = self.repository.get(inventory_number)
```

```
        if device is None:
            raise KeyError(inventory_number)
        return device

class DevicePayload(TypedDict):
    inventory_number: str
    device_type: str
    manufacturer: str
    model: str
    cost: str

def from_payload(payload: DevicePayload) -> Device:
    return Device(
        payload["inventory_number"],
        payload["device_type"],
        payload["manufacturer"],
        payload["model"],
        Decimal(payload["cost"]),
    )
```

Registry підтримує len(), for, in та [] через __len__, __iter__, __contains__, __getitem__. Money має __str__ та __add__. TypedDict описує вхідний словник, а from_payload явно створює перевірений Device. Це відокремлює формат передавання від доменної поведінки.

7. Демонстраційний запуск та експеримент типізації

```
Devices: 2 Total: 48000.00 UAH Average: 24000.00 UAH
Computer maintenance: 180 days
Printer maintenance: 90 days
Contains C-1: True
Best: Computer(inventory_number='C-1', device_type='Computer', manufacturer='Dell',
model='Latitude', cost=Decimal('40000'), ram_gb=16)
Dell: [Computer(inventory_number='C-1', device_type='Computer', manufacturer='Dell',
model='Latitude', cost=Decimal('40000'), ram_gb=16)]
```

```
# Copy this to a temporary .py file and run mypy; not production code.
from computer_inventory.domain import InMemoryRepository, Manufacturer
from computer_inventory.models import Device
inventory_number: int = "INV-001"
def total() -> int:
    return "100"
repository: InMemoryRepository[Device] = InMemoryRepository()
repository.add(Manufacturer("Dell"))
```

```
work\type_errors.py:4: error: Incompatible types in assignment (expression has type "str",
variable has type "int")  [assignment]
work\type_errors.py:6: error: Incompatible return value type (got "str", expected "int")
[return-value]
work\type_errors.py:8: error: Argument 1 to "add" of "InMemoryRepository" has incompatible
type "Manufacturer"; expected "Device"  [arg-type]
Found 3 errors in 1 file (checked 1 source file)
```

Навмисний приклад із трьома помилками винесено за межі production-коду. Виправлення: inventory_number має тип str, total повертає int, a repository.add отримує Device. Правильні доменні модулі проходять муру; фінальна перевірка всіх 16 source-файлів також успішна.

```
Success: no issues found in 16 source files
```

8. Порівняння способів проєктування

Рішення	**Зв'язність**	**Тестування та заміна**
Сервіс сам створює concrete repository	Висока залежність від способу збереження	Потрібні patch або зміна сервісу
DI через InventoryRepository	Залежність від вузького контракту	Можна передати InMemoryRepository або Mock
Успадкування Computer/Printer від Device	Спільні поля й інваріанти пристрою	Доречне відношення is-a
Композиція registry + repository + policy	Незалежна зміна збереження й політик	Розширення без розростання ієрархії

Для обробника CSV композиція reader + validator + exporter дозволяє незалежно змінювати формат і правила; окрема ієрархія CsvProcessor для кожної комбінації спричинила б більше зв'язаних підкласів. У цьому проєкті успадкування застосовано до різновидів техніки, а композицію — до сервісних залежностей.

Код і історія лабораторних: https://github.com/dimakaparnyk/professional-python-labs. Для відповідного етапу використовується тег lab-04, фінальна інтегрована реалізація — main (перевірений CI commit 2e3fb3d).

9. Відповіді на контрольні запитання

1. Що таке class?

Опис типу об'єктів із їхніми атрибутами та поведінкою.

2. Що таке object?

Конкретна сутність у пам'яті, яка має тип, стан та ідентичність.

3. Що таке instance?

Екземпляр певного класу.

4. Для чого __init__()?

Для ініціалізації вже створеного екземпляра; саме створення виконує __new__().

5. Що таке instance attribute?

Атрибут, що належить окремому екземпляру.

6. Що таке class attribute?

Атрибут класу, доступний спільно екземплярам, якщо його не перекрито.

7. Що таке encapsulation?

Об’єднання стану й поведінки та контроль способів зміни внутрішніх даних.

8. Для чого properties?

Для доступу до обчислюваних або контрольованих значень як до атрибутів.

9. Чим property відрізняється від public field?

Property викликає getter або setter і може перевіряти дані чи обчислювати результат.

10. Що таке inheritance?

Побудова спеціалізованого класу на основі іншого з успадкуванням його контракту.

11. Що означає relation is-a?

«Є різновидом»: Computer є Device і повинен відповідати контракту пристрою.

12. Що таке composition?

Побудова об’єкта з інших об’єктів, поведінку яких він використовує.

13. Що означає relation has-a?

«Має»: InventoryRegistry має repository, а сервіс має maintenance policy.

14. Коли composition краща за inheritance?

Коли потрібно незалежно замінювати поведінку або залежності без розширення ієрархії типів.

15. Для чого super()?

Для виклику наступної реалізації за порядком MRO, зокрема перевірок базового Device.

16. Що таке polymorphism?

Можливість однаково працювати з об’єктами різних конкретних типів через спільний контракт.

17. Що таке duck typing?

Використання об’єкта за підтримуваними операціями, а не лише за назвою його класу.

18. Що таке ABC?

Abstract Base Class — базовий клас, що задає обов’язкові абстрактні операції.

19. Для чого abstractmethod?

Позначає метод, який конкретний нащадок повинен реалізувати для створення екземплярів.

20. Чи можна створити instance abstract class?

Ні, доки хоча б один abstractmethod лишається нереалізованим.

21. Що таке Protocol?

Опис структурного контракту для статичної перевірки сумісності об’єктів.

22. Чим Protocol відрізняється від ABC?

Protocol перевіряє наявність потрібних членів структурно; ABC зазвичай спирається на явне успадкування.

23. Що таке structural typing?

Сумісність типів за їхньою структурою — набором методів і атрибутів.

24. Що таке nominal typing?

Сумісність за оголошеним зв’язком класів або іменами типів.

25. Що таке dataclass?

Клас даних, для якого декоратор генерує типові службові методи.

26. Які methods генерує dataclass?

За налаштуваннями генеруються __init__, __repr__, __eq__, а інші методи залежать від параметрів.

27. Для чого default_factory?

Створює новий початковий об’єкт для кожного екземпляра, наприклад окремий список.

28. Чому mutable default є проблемою?

Один спільний змінюваний об’єкт може випадково використовуватися кількома екземплярами.

29. Для чого __post_init__()?

Для перевірок та похідної ініціалізації після згенерованого __init__.

30. Що означає frozen=True?

Забороняє звичайне переприсвоєння полів екземпляра; це не глибока незмінність вкладених об’єктів.

31. Що означає slots=True?

Створює slots для полів і зазвичай прибирає словник довільних атрибутів екземпляра.

32. Що таке dunder method?

Спеціальний метод із подвійними підкресленнями, який реалізує протокол Python.

33. Для чого __repr__()?

Для однозначного діагностичного представлення об’єкта.

34. Для чого __str__()?

Для зрозумілого користувачу текстового представлення.

35. Для чого __len__()?

Для підтримки len(object).

36. Для чого __iter__()?

Для повернення ітератора об’єкта.

37. Для чого __contains__()?

Для операції перевірки належності item in object.

38. Для чого __call__()?

Для можливості викликати екземпляр як функцію.

39. Що таке SOLID?

П’ять принципів об’єктного проєктування: SRP, OCP, LSP, ISP, DIP.

40. Що означає SRP?

Одна сутність повинна мати одну основну причину змінюватися.

41. Що означає OCP?

Поведінку бажано розширювати через нові реалізації без зміни стабільного клієнтського коду.

42. Що означає LSP?

Підтип має бути взаємозамінним із базовим типом без порушення його контракту.

43. Що означає ISP?

Клієнтам слід залежати від вузьких потрібних інтерфейсів.

44. Що означає DIP?

Високорівнева логіка має залежати від абстракцій, а не конкретних адаптерів.

45. Що таке dependency injection?

Передавання об’єкту готових залежностей, зазвичай через конструктор.

46. Чому DI покращує testability?

У тесті можна передати fake або Mock замість БД чи зовнішнього сервісу.

47. Що таке type hint?

Анотація очікуваного типу значення, параметра або результату.

48. Чи перевіряє Python type hints автоматично в runtime?

Ні; для перевірок виконання потрібна окрема валідація.

49. Що таке static type checker?

Інструмент аналізу сумісності типів без запуску бізнес-сценарію.

50. Для чого mypy?

Для статичної перевірки анотацій та пошуку несумісних аргументів, присвоєнь і результатів.

51. Для чого Pyright?

Для статичної перевірки типів і підтримки редакторів із швидким аналізом коду.

52. Що таке TypeVar?

Змінна типу, що зв’язує узгоджений тип у кількох частинах контракту.

53. Що таке Generic?

Базовий механізм оголошення параметризованих типів.

54. Що означає Generic[T]?

Клас приймає конкретний параметр типу T, наприклад InMemoryRepository[Device].

55. Що таке bound TypeVar?

Змінна типу з верхньою межею: T повинен бути сумісним із заданим базовим типом.

56. Що таке constrained TypeVar?

Змінна типу, обмежена конкретним переліком допустимих типів.

57. Що таке TypedDict?

Опис типів конкретних іменованих ключів звичайного словника.

58. Чим TypedDict відрізняється від dataclass?

TypedDict перевіряє структуру dict статично; dataclass створює окремий клас з атрибутами й методами.

59. Коли доцільно використовувати TypedDict?

Для опису JSON-подібних словників і зовнішніх payload із відомими полями.

60. Чому external payload і domain object краще розділяти?

Щоб формат обміну не визначав внутрішню модель, а перевірки й перетворення виконувалися на межі системи.

10. Висновок

Створено типізовану предметну модель з незмінюваними значеннями, власною колекцією, абстрактною політикою обслуговування та ін’єкцією репозиторію. Муру виявив три навмисні порушення типів; робочі модулі проходять перевірку.